

Using Kernel Density Function and Student's t-distribution to model Value at Risk and Expected Shortfall using Monte Carlo method

Gianni Mazaud
gianni.mazaud@icloud.com

Yuto Murata

Abstract—Basel III regulation imposes a clear market risk frameworks for financial institutions. The principal shift rests in the core of the attention now turned toward a 97.5% Expected Shortfall, rather than VaR at 99% via FRTB.

In this paper we use Monte Carlo simulations through Student's t-distribution and non-parametric Kernel Density Estimator (KDE) to calculate VaR and CVaR at the 99% and 97.5% threshold.

Empirically, our simulations demonstrate that the Geometric Brownian Motion framework penalizes risk architecture by collapsing the threshold difference between Value at Risk and Expected Shortfall. In contrast, both the parametric Student's t-distribution and the non-parametric Kernel Density Estimator successfully capture extreme market shocks with fatter tail distributions. In each scenario, we observe that $CVaR(97.5\%)$ is greater than $VaR(99\%)$, validating the Basel III Fundamental Review of the Trading Book (FRTB) guidelines.

We demonstrate empirically that KDE captures the asymmetric tail structure of real returns that Student's t cannot, with implications for Basel III capital adequacy.

Index Terms—Value at Risk, Expected Shortfall, Monte Carlo modeling, Kernel Density Estimator, Student's t-distribution, Asymmetric tail risk

I. INTRODUCTION

Value at Risk is defined as: "a measure of the worst expected loss on a portfolio of instruments resulting from market movements over a given time horizon and a pre-defined confidence level" [1] [2]. Formally, it can be written as follow:

$$VaR_\alpha = \inf \{l \in \mathbb{R} : \mathbb{P}(L > l) \leq 1 - \alpha\}$$

Expected Shortfall or Conditional Value at Risk is defined as: "a measure of the average of all potential losses exceeding the VaR at a given confidence level" [1]. Mathematically, we can write it as:

$$CVaR_\alpha = \mathbb{E}[L|L \geq VaR_\alpha]$$

Under the Basel III regulation, the confidence level for VaR is $\alpha = 99\%$, even though it focuses primarily on Expected Shortfall. Developed by the Basel Committee on Banking Supervisions (BCBS), this regulation was first implemented in July 2013 and finally reformed in December 2017. Its first aim was to respond to the 2007-2008 global financial crisis. To counter these risks it mainly focuses on enhanced capital

requirements, non-risk-based leverage ratio, and counterparty credit risk.

In terms of market risk management, Basel III —specifically through the Fundamental Review of the Trading Book (FRTB)— shifts the core metric from a 99% Value at Risk (VaR) to a 97.5% Expected Shortfall (ES). This shift implies that regulators no longer satisfied with knowing just the threshold of a bank's potential trading losses, but rather the actual severity of losses when a tail-risk event occurs. Additionally, while the previous VaR framework assumed a static, short-term liquidity horizon across all asset classes, the new CVaR-based framework forces banks to map different liquidity horizons to different risk factors.

Banks aren't talkative on the subject and it's hard to really understand, when they are using the Monte Carlo method, which diffusion model they use [3]. Our goal here will be to compare three different approaches. First, we will use a Geometric Brownian motion, that assumes the logarithm of asset prices follows a Brownian motion with a drift. It implies that the probability of extreme events is highly underestimated. In order to offset that, a Student's t-distribution will be implemented to take into account fat-tail risks. This model is usually a better fit for financial returns and it captures extreme events more accurately. Finally, we are going to use a Kernel Density Estimator which is a non-parametric way to estimate the true distribution of financial returns directly from data.

II. USUAL METHODS

Parametric method

Generally, Basel III allows for three methods: parametric, historic, and Monte Carlo.

Parametric method simply assumes loss follows a Gaussian (normal) distribution, then we have: $L \sim N(\mu_L, \sigma_L^2) \Rightarrow L = \mu_L + \sigma_L U$, with $U \sim N(0, 1)$.

Under this method, the formulas are:

$$VaR_\alpha = \mu_L + \sigma_L \Phi^{-1}(\alpha)$$

Applying this to our definition of Expected Shortfall, it implies:

$$CVaR_\alpha = \mathbb{E}[L|L \geq VaR_\alpha]$$

$$= \frac{1}{1-\alpha} \int_{VaR_\alpha}^{+\infty} l f_L(l) dl \mu_L + \sigma_L \frac{\varphi(\Phi^{-1}(\alpha))}{1-\alpha}$$

(See Appendix 1. for demonstrations.)

Historical method

This method rests on the assumption that the worst case scenarios of the period taken into account will remain the same. Nevertheless, reading [3], we can see that the distribution can still be adapted following some scenarios. Furthermore, it assumes portfolio remains the same, the weight of each asset being unchanged.

Let $r_{i,t}$ be the daily returns for each asset i at date $t \in [i, T]$ our historical interval. We apply today's weight w to each historical return scenario:

$L_t = -\sum_{i=1}^T \omega_i r_{i,t} = -w^T r_t$ (it takes the opposite value since loss is considered the opposite of the asset return). It gives us T realized loss scenarios, using the current portfolio composition replayed through history. We order the T losses in ascending order: $L_{(1)} < L_{(2)} < \dots < L_{(T)}$.

Finally, we have:

$$VaR_\alpha = L_{[\alpha, T]}$$

$$CVaR_\alpha = \frac{1}{T - \alpha T} \sum_{t=\alpha T+1}^T L_{(t)}$$

III. MONTE CARLO METHOD

Method framework

The Monte Carlo method is a way to find answers to hard financial problems by using random numbers and simulating thousands of independent paths following the same distribution. First, we choose a statistical rule for how our assets behave (for instance a log-normal distribution or a Student's t-distribution). It allows us to see where the stock prices can potential go. Finally, we put all these simulated results together from the worst loss to the biggest profit. This gives us a huge list of possibilities, and we just look at the bottom 1% of the worst results to easily find the Value at Risk (VaR) or take the average of those bad results to get the Conditional VaR (CVaR).

Statistical distributions

Our goal now is to implement and compare three different statistical distributions: log-normal, Student's t, and Kernel Density Estimator.

Log-normal distribution

Geometric Brownian motion makes sure prices are strictly positive by using an exponential. Also, it scales volatility and drift based on the current price. Finally, this model assumes that continuously compounded returns are normally

distributed. If we assume that an asset S follows a log-normal distribution, we have:

$$dS_t = \mu S_t dt + \sigma S_t dW_t$$

Where S_t is the asset price at time t , μ the drift (expected return), σ the volatility coefficient, dt an infinitesimal time step, dW_t a Wiener process (or Brownian motion) increment, which captures the random market shocks where $dW_T \sim N(0, dt)$ [4]. It has the following properties:

- 1) $W_0 = 0$.
- 2) It has independent increments.
- 3) For any two time steps s and t (where $t > s$), the change in value is normally distributed: $W_t - W_s \sim N(0, t - s)$.

The analytical solution, by applying Itô's Lemma [5] to solve the SDE, gives:

$$S_T = S_0 \exp \left(\left(\mu - \frac{\sigma^2}{2} \right) T + \sigma W_T \right)$$

(Demonstrations are shown in Appendix 2.)

If we set $s = 0$ and look strictly at a single future maturity date $t = T$, the total increment is:

$$W_T - W_0 = W_T \sim N(0, T)$$

If we scale a standard normal random variable $Z \sim N(0, 1)$ by multiplying it by $\sqrt{T}$, its mean remains 0 and its variance becomes $(\sqrt{T})^2 = T$. Therefore:

$$W_T \stackrel{d}{=} Z\sqrt{T}$$

Since W_T can be written as $Z\sqrt{T}$, where $Z \sim N(0, 1)$, the formula used is:

$$S_T = S_0 \exp \left(\left(\mu - \frac{\sigma^2}{2} \right) T + \sigma Z\sqrt{T} \right)$$

Student's T-distribution [6]

The Student's t-distribution does not only rely on the mean and variance, but introduces a third parameter: degrees of freedom (ν). Further in this paper, we use $\nu = 4$ as a conservative choice to ensure fat-tailed behavior while maintaining finite variance. The mathematical formula for the Probability Density Function (PDF) of such a distribution is:

$$f(x) = \frac{\Gamma\left(\frac{\nu+1}{2}\right)}{\sqrt{\nu\pi}\Gamma\left(\frac{\nu}{2}\right)} \left(1 + \frac{x^2}{\nu}\right)^{-\frac{\nu+1}{2}}$$

Where the Gamma function is defined as:

$$\Gamma(z) = \int_0^{+\infty} t^{z-1} e^{-t} dt$$

If the degrees of freedom is low ν the distribution has very fat tails and a higher peak (leptokurtic). However when $\nu \rightarrow \infty$ the distribution mathematically converges into the standard normal distribution.

Modeling an asset using this distribution, we get:

$$S_T = S_0 \exp \left(\mu T + \sigma \sqrt{\frac{\nu-2}{\nu}} T_\nu \sqrt{T} \right)$$

Where T_ν is a random variable drawn from a standard Student's t-distribution with ν degrees of freedom and $\sqrt{\frac{\nu-2}{\nu}}$ is the scaling factor ensuring that the variance of the return over time matches our target volatility. (Proof is given in Appendix 3.)

Kernel Density Estimator

The formal mathematical formula for a Kernel Density Estimator is:

$$\hat{f}_h(x) = \frac{1}{nh} \sum_{i=1}^n K\left(\frac{x - x_i}{h}\right)$$

Where h is the bandwidth, and $K(\cdot)$ the Kernel function. If we use the Gaussian (Normal) Kernel: $K(u) = \frac{1}{\sqrt{2\pi}} e^{-\frac{1}{2}u^2}$, we finally get:

$$\hat{f}_h(x) = \frac{1}{nh\sqrt{2\pi}} \sum_{i=1}^n e^{-\frac{1}{2}\left(\frac{x-x_i}{h}\right)^2}$$

the summation gives us exactly the probability density function (PDF) of a normal distribution $N(x_i, h^2)$. Therefore, the KDE can be written as:

$$\hat{f}_h(x) = \frac{1}{n} \sum_{i=1}^n f_N(x|\mu = x_i, \sigma^2 = h^2)$$

This tells us that the KDE treats the asset return distribution as a uniform mixture of n distinct normal distributions. Assuming that $y_j \sim N(x_i, h^2)$ we have that:

$$y_j = x_i + hZ_j$$

To decide which value to choose for h we will use the Silverman's Rule of Thumb [7], that states:

$$h = \left(\frac{4\hat{\sigma}^5}{3n}\right)^{\frac{1}{5}} \approx 1.05\hat{\sigma}n^{-\frac{1}{5}}$$

The core difference of this approach compared to a Brownian motion is that it assume no prior functional form for the distribution. Instead of forcing the data into a bell curve, the data defines its own shape.

How to decide which distribution model to use?

KDE (Non-Parametric) does not assume any underlying shape for our data. It builds a distribution directly from historical data points by placing a "kernel" (smooth curve) over each observation. It is purely empirical. Student's t (Parametric) assumes a fixed mathematical shape defined by specific parameters (mean, scale, and degrees of freedom). It is symmetrical but features heavy tails, making it excellent for capturing extreme financial shocks. GBM (Stochastic Process) unlike the other two, GBM is not a static probability distribution—it is a time-dependent path generator. It models how an asset price moves over time based on a constant drift (trend) and volatility (random walk).

IV. IMPLEMENTATIONS & RESULTS

Using Python, we can simulate an asset N times and adjust the characterizing parameters as required for our study. The python code can be found in Appendix 4. For our study, we construct an equally-weighted portfolio made of: S&P 500, NASDAQ, CAC40, FTSE 100, DAX, and Hang Seng. This portfolio is of course a simplified version of what could be seen in practice but we assume for a CVaR and VaR simulation it can fit. We extract the data for a five years period, from 06/01/2021 to 06/01/2026, with yfinance module in Python. The prices used are daily close prices and we consider log-returns since it assumes symmetry around zero which is useful for modeling. Mean and standard deviation are computed via a Maximum Likelihood estimation which provides a uniform, consistent framework to compare the normal distribution against non-Gaussian models (like the Student's t-distribution) that require statistical fitting rather than simple averages. After gaining these data, we model 50,000 paths for each distribution. This presents some assumptions and simplifications. In fact, real trading books re-balance often which is not the case here in our model. To get a better picture of how returns are distributed, here are the histograms representing the distribution of our generated-data.

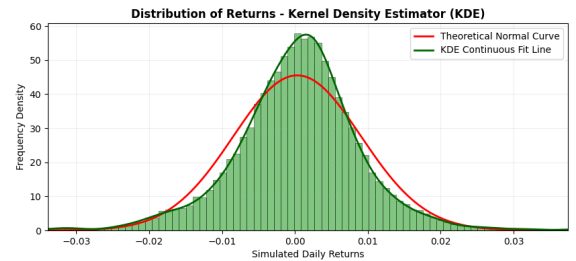
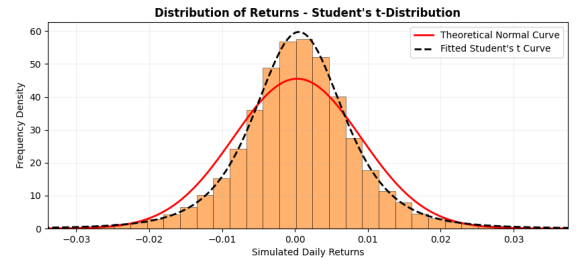
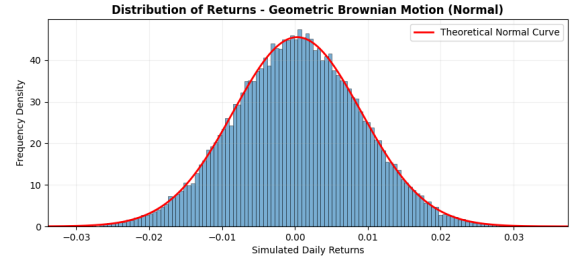


TABLE I: Statistical Significance Tests for Skewness and Kurtosis

Model	Test / Metric	Value / Stat	z-score	p-value
GBM	Skewness	-0.0095	-0.8703	0.384119
	Ex. Kurtosis	+0.0010	+0.0465	0.962946
Student's t	Skewness	+0.0004	+0.0330	0.973636
	Ex. Kurtosis	+14.7932	+675.2208	0.000000***
KDE	Skewness	-1.0377	-94.7309	0.000000***
	Ex. Kurtosis	+11.2080	+511.5783	0.000000***

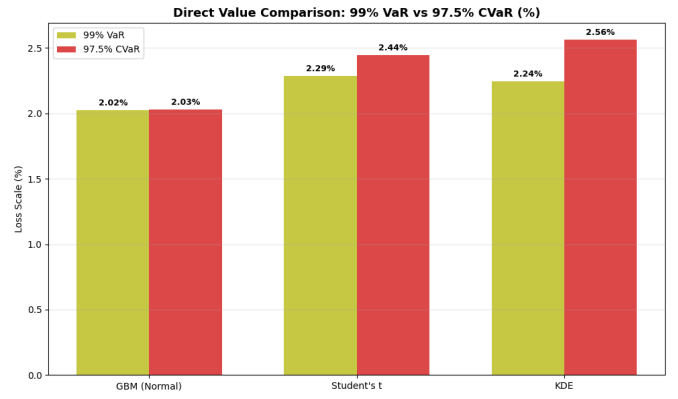
Significance codes: *** $p < 0.001$, ** $p < 0.01$, * $p < 0.05$

The Geometric Brownian Motion (GBM) model assumes that asset returns follow a perfect normal distribution. In our simulation, the GBM skewness yields a z-score of -0.8703 ($p = 0.384$), and the excess kurtosis yields a z-score of $+0.0465$ ($p = 0.963$). Because both p-values are much larger than 0.05, these results are not statistically significant, meaning the distribution is perfectly Gaussian. Consequently, the GBM model is completely blind to unexpected market shocks or severe tail risks, making it an unrealistic tool for downside risk management.

To fix the limitations of the normal model, a Student's t-distribution was fitted to the data. While the skewness remains statistically insignificant ($z = +0.0330$, $p = 0.974$), the excess kurtosis shifts drastically, showing an extreme z-score of $+675.2208$ ($p < 0.001$). This highly significant p-value proves that the asset's returns have much fatter tails than a normal distribution. On the chart, the Student's t curve successfully captures the heavy risk in the tails, making it a much more reliable model for calculating portfolio Value-at-Risk (VaR).

Although the Student's t-distribution models fat tails well, its mathematical structure forces the data to be perfectly symmetric. The non-parametric Kernel Density Estimator (KDE) removes this restriction and reveals a highly significant negative skewness with a z-score of -94.7309 ($p < 0.001$), alongside a significant excess kurtosis z-score of $+511.5783$ ($p < 0.001$). These significant p-values confirm that the distribution is heavily distorted to the left. This proves the true asymmetry of financial risk: asset returns typically face sudden, severe negative crashes rather than positive jumps.

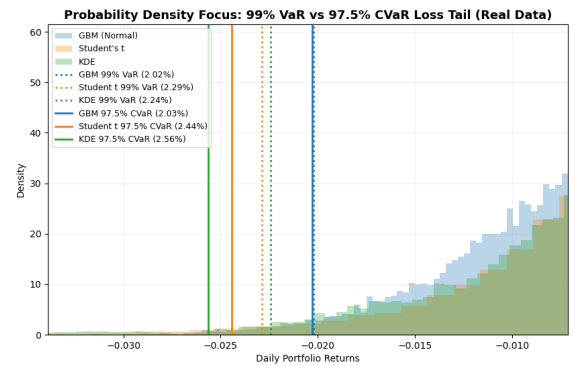
Finally, putting altogether our estimated risk measures:



This histogram shows a big difference between the models. Under the normal GBM model, the $VaR(99\%) = 2.02\%$ and the $CVaR(97.5\%) = 2.03\%$ are almost exactly the same, with a gap of only 0.01%. This happens because the normal model assumes that extreme losses are nearly impossible. In the real world, using this normal model is dangerous because it leaves a portfolio unprotected against big market drops.

When we look at the other models, the risk numbers increase to show the true dangers of the market. The Student's t-distribution gives a $VaR(99\%)$ of 2.29% and a $CVaR(97.5\%)$ of 2.44%, showing a larger gap of 0.15% because it captures fat tails. However, the KDE model gives the most realistic view of risk. Even though its $VaR(99\%)$ is 2.24%, its $CVaR(97.5\%)$ jumps to the highest level at 2.56%. This 0.32% gap is caused by the negative skewness of the data, proving that real-world market crashes are much worse than simple models predict.

Also, we notice that in every case $CVaR(97.5\%)$ is above $VaR(99\%)$ reflecting the reinforcement in capital exposition regulation imposed by Basel III.

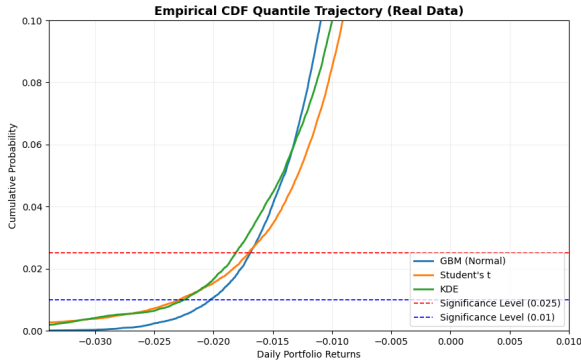


The graph of Probability Density shows the worst-case losses for the three different models. The normal model (GBM) is represented by the blue lines. Here, the $VaR(99\%)$ is 2.02% and the $CVaR(97.5\%)$ is 2.03%. Because these two lines are almost on top of each other, the model assumes that

there are no risk of extreme scenarios.

The other two models look at the actual danger in the market much better. As seen in the graph, the Student's t distribution (orange lines) separates the VaR (2.29%) and CVaR (2.44%) because it takes into consideration the market has fat tails. However, the KDE model (green lines) shows the most realistic danger. Even though its $VaR(99\%)$ is 2.24%, its $CVaR(97.5\%)$ moves way to the left at 2.56%. This large gap happens because real financial data has a negative skew, meaning that when markets crash, they fall fast and deep.

$CVaR(97.5\%)$ is always represented at the left (deeper into high-loss territory). Unlike the normal distribution, KDE and Student's t-histogram bars continue to exist on the left part of the graph. This illustrates again the reason why Basel III guidelines transitioned regulatory capital calculations to a $CVaR(97.5\%)$ framework.



The chart empirical CDF shows the cumulative probability of negative return. The two horizontal dashed lines represent the risk thresholds: the red line is the 2.5% risk level (used for 97.5% CVaR), and the blue line is the 1% risk level (used for 99% VaR). This chart shows exactly where each model intersects these risk lines.

As we look more closely, we can see how the lines cross the thresholds differently.

- The blue GBM (blue) line stays to the right of the others.
- However, both the Student's t (orange) and KDE (green) lines are much further to the left.

The fact that the orange and green curves sit higher up on the left side proves that large losses happen much more often in real life than the normal GBM model expects.

V. LIMITATIONS

Our entire simulation operates on a one-day return horizon. FRTB explicitly requires liquidity-adjusted horizons ranging from 10 days to 120 days. Our results therefore represent a best-case, most liquid scenario and would need to be scaled before being operationally used in a real regulatory context.

Our Monte Carlo draws each scenario as if the six indices move independently. In reality, tail dependence between equity

markets is well documented: during the 2020 Covid crash or 2008 crisis, global indices fell simultaneously and correlations spiked toward 1. A copula-based approach would capture this.

The equal-weight assumption is held fixed throughout the whole time-period where we collect data. In reality, portfolio allocation evolves continuously which changes the loss distribution.

Silverman's rule-of-thumb is considered optimal for estimating the bulk of a distribution, but also it is known to over-smooth the tails. A smaller bandwidth would produce heavier tails and at the same time higher risk measures estimates.

The Student's t-distribution is symmetric by construction, meaning it assigns equal probability to extreme gains and extreme losses. Since financial returns exhibit asymmetric crash risk (negative skewness), the Student's systematically underestimates left-tail severity relative to a model that allows for skewness.

VI. BACKTESTING

Using data from 2015 to 2024, we can split the sample between training period and test period. Here we divide it from 2015 to 2020 and the second part is from 2021 to 2024. Here are the results:

TABLE II: VaR/CVaR Estimates (fitted on 2015–2020)

Model	VaR(99%)	CVaR(97.5%)
GBM	2.43%	2.43%
Student t	2.89%	3.36%
KDE	3.21%	3.63%

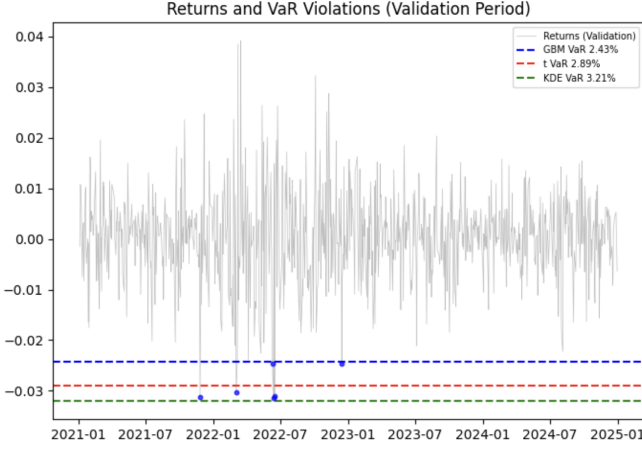
TABLE III: Backtest: VaR(99%) Violation Rates (2021–2024)
Theoretical target: 1.00% violation rate

Model	Violation Rate	Judgment
GBM	0.64%	lax
Student t	0.43%	slightly conservative
KDE	0.00%	slightly conservative

We again observe that GBM underestimates tail risks, both measures being equal. Half of the black swan events result in a breach. KDE's 0.00% violation rate suggests a slightly conservatism in practice. A 0% violation rate means excess capital allocation, which is inefficient from a regulatory standpoint. Student's t-distribution appears most balanced between accuracy and conservatism. Black swan events (COVID: -7.58%) exceeded all three models — no parametric or non-parametric approach can fully capture them. However, KDE approach is the most robust approach to extreme scenarios like these ones.

TABLE IV: Key Event Analysis

Event	Return	GBM	Student t	KDE
COVID crash (2020/3)	-7.58%	Breach	Breach	Breach
Rate hike shock (2022/6)	-3.15%	Breach	Breach	OK
Russia invasion (2022/2)	-1.73%	OK	OK	OK
SVB collapse (2023/3)	-1.07%	OK	OK	OK



VII. CONCLUSION

This paper introduces three distributions to evaluate Value at Risk and Expected Shortfall under the Basel III/FRTB regulation. We modeled 50,000 different random paths for each distribution in order to evaluate these risk measures. The first distribution used is a geometric Brownian motion (using Gaussian distribution assumptions), and our findings show that it fails to capture fat-tail risk. Indeed, as calculated, the excess kurtosis is near zero showing the lack of consideration for extreme scenarios.

The second distribution implemented is the Student's t -distribution in order to overcome that issue of not considering fat-tail risk. This distribution exhibits a clearly positive excess kurtosis which implies a leptokurtic distribution as we can observe with the corresponding histogram. Finally, the Kernel Density Estimator also exhibits a leptokurtic distribution and a clearly lower skewness than the two others. As the data tells, this distribution gives the highest Expected Shortfall but still has a lower Value at Risk, relative to the thresholds imposed by the regulation. This model is different in the fact that it is non-parametric, it let the data defines its own form.

Our findings demonstrate the danger of normality assumption. As we showed, the Gaussian distribution does not take consideration of fat-tail risks that are essential in risk measurement. Backtesting helps at seeing the weakness of this distribution since it is the one with the highest violation rate. On the one side, data clearly shows the geometric Brownian motion exhibits lower values for both measures (Expected Shortfall and Value at Risk) and gives almost the value for both of them showing again the ignorance of extreme scenarios. On the other side, KDE as well as Student's t distributions have a

much higher CVaR(97.5%) than VaR(99%) proving the utility of the Basel III regulation that shifts toward the first one in order to reinforce capital requirement.

A parametric approach assumes that the distribution of a portfolio returns follows a known statistical distribution equation. To capture fat tails with a parametric model, Gaussian distribution must be forgotten. In a non-parametric framework like the Kernel Density Estimator (KDE), we get rid of equations to use data directly. Then, it captures leptokurtosis naturally. This gives more accurate measures, especially for hidden structural risks (such as the negative skewness caused by market sell-offs).

REFERENCES

- [1] Basel Committee on Banking Supervision, "Explanatory note on the minimum capital requirements for market risk," 2019. [Online]. Available: https://www.bis.org/bcbs/publ/d457_note.pdf
- [2] J.-P. Laurent, "Modèle structurel du risque de crédit," http://laurent.jeanpaul.free.fr/Enseignement/GRF_2023-24_modele_structurel_du_risque_de_credit.pdf, 2023, accessed: 2026-06-04.
- [3] Société Générale Group, "2025 universal registration document: 2024 annual financial report," Société Générale, Universal Registration Document Form 297, March 2025, filed with the Autorité des Marchés Financiers (AMF) on March 12, 2025. [Online]. Available: <https://usprogram.socgen.com/files/297.pdf>
- [4] M. Csörgő, "Brownian motion-wiener process," *Canadian Mathematical Bulletin*, vol. 22, no. 3, pp. 257–279, 1979. [Online]. Available: <https://www.cambridge.org/core/journals/canadian-mathematical-bulletin/article/brownian-motion-wiener-process/17DB3F7FE5C4F6216E6A214D166D44A6>
- [5] K. Itô, "On stochastic differential equations," *Memoirs of the American Mathematical Society*, vol. 4, pp. 1–51, 1951, cited by: 3105. [Online]. Available: <https://doi.org/10.1090/memo/0004>
- [6] Student, "The probable error of a mean," *Biometrika*, vol. 6, no. 1, pp. 1–25, 1908.
- [7] B. W. Silverman, *Density Estimation for Statistics and Data Analysis*, ser. Monographs on Statistics and Applied Probability. London: Chapman and Hall, 1986, vol. 26.

VIII. APPENDICES

Appendix 1. — demonstrations for parametric method

VaR is such that [2]:

$$\mathbb{P}(L < VaR_\alpha) = \alpha$$

Since we assumed $L \sim N(\mu_L, \sigma_L)$:

$$\Leftrightarrow \mathbb{P}(\mu_L + \sigma_L U < VaR_\alpha) = \alpha$$

With $U \sim N(0, 1)$, We can simplify as:

$$\Leftrightarrow \mathbb{P}\left(U < \frac{VaR_\alpha - \mu_L}{\sigma_L}\right) = \alpha$$

$$\Leftrightarrow \Phi\left(\frac{VaR_\alpha - \mu_L}{\sigma_L}\right) = \alpha$$

$$\Leftrightarrow \frac{VaR_\alpha - \mu_L}{\sigma_L} = \Phi^{-1}(\alpha) \Leftrightarrow VaR_\alpha = \mu_L + \sigma_L \Phi^{-1}(\alpha)$$

Applying this to our definition of Expected Shortfall, it implies:

$$CVaR_\alpha = \mathbb{E}[L|L \geq VaR_\alpha] = \frac{1}{1-\alpha} \int_{VaR_\alpha}^{+\infty} l f_L(l) dl$$

with f_L the probability density function of $L \sim N(\mu_L, \sigma_L^2)$.

Hence:

$$f_L(l) = \frac{1}{\sqrt{2\pi\sigma_L^2}} \exp\left(-\frac{(l-\mu_L)^2}{2\sigma_L^2}\right)$$

Let

$$\begin{aligned} \varphi : \frac{l-\mu_L}{\sigma_L} &\rightarrow \frac{1}{\sqrt{2\pi}} \exp\left(-\frac{\left(\frac{l-\mu_L}{\sigma_L}\right)^2}{2}\right) \\ &= \frac{1}{\sqrt{2\pi}} \exp\left(-\frac{(l-\mu_L)^2}{2\sigma_L^2}\right) \end{aligned}$$

This gives us:

$$f_L(l) = \frac{1}{\sigma_L} \varphi\left(\frac{l-\mu_L}{\sigma_L}\right)$$

Finally, we can compute the Expected Shortfall:

$$CVaR_\alpha(L) = \frac{1}{1-\alpha} \int_{VaR_\alpha(L)}^{+\infty} \frac{l}{\sigma_L} \varphi\left(\frac{l-\mu_L}{\sigma_L}\right) dl$$

Let's define:

$$z = \frac{l-\mu_L}{\sigma_L} \Leftrightarrow l = \mu_L + \sigma_L z \Rightarrow dl = \sigma_L dz$$

This gives us:

$$\frac{1}{\sigma_L} \varphi\left(\frac{l-\mu_L}{\sigma_L}\right) \sigma_L dz = \varphi(z) dz$$

$$VaR_\alpha = \mu_L + \sigma_L \Phi^{-1}(\alpha)$$

$$z = \frac{l-\mu_L}{\sigma_L} = \frac{(\mu_L + \sigma_L \Phi^{-1}(\alpha) - \mu_L)}{\sigma_L} = \Phi^{-1}(\alpha)$$

Finally:

$$CVaR_\alpha(L) = \frac{1}{1-\alpha} \int_{\Phi^{-1}(\alpha)}^{+\infty} (\mu_L + \sigma_L z) \varphi(z) dz$$

$$CVaR_\alpha = \frac{1}{1-\alpha} \left(\mu_L \int_{\Phi^{-1}(\alpha)}^{+\infty} \varphi(z) dz + \sigma_L \int_{\Phi^{-1}(\alpha)}^{+\infty} z \varphi(z) dz \right)$$

Let's compute separately these two integrals:

$$I_1 = \int_{\Phi^{-1}(\alpha)}^{+\infty} \varphi(z) dz = 1 - \Phi(\Phi^{-1}(\alpha)) = 1 - \alpha$$

$$I_2 = \int_{\Phi^{-1}(\alpha)}^{+\infty} z \varphi(z) dz$$

$$= - \int_{\Phi^{-1}(\alpha)}^{+\infty} \varphi'(z) dz = - \left[\varphi(z) \right]_{\Phi^{-1}(\alpha)}^{+\infty} = \varphi(\Phi^{-1}(\alpha))$$

We use here two arguments:

- $\varphi'(z) = -z\varphi(z)$

- $\varphi(z) \rightarrow 0$ when $z \rightarrow +\infty$

Hence:

$$CVaR_\alpha = \frac{1}{1-\alpha} [\mu_L(1-\alpha) + \sigma_L \varphi(\Phi^{-1}(\alpha))]$$

$$\Leftrightarrow CVaR_\alpha = \mu_L + \sigma_L \frac{\varphi(\Phi^{-1}(\alpha))}{1-\alpha}$$

Appendix 2. — Brownian motion

We let the following formula:

$$dS_t = \mu S_t dt + \sigma S_t dW_t$$

Let: $f(S_t) = \ln(S_t)$. For a continuous function $f(t, S_t)$ the second-order Taylor expansion using Itô's Lemma [5] gives us:

$$df = \frac{\partial f}{\partial t} dt + \frac{\partial f}{\partial S} dS_t + \frac{1}{2} \frac{\partial^2 f}{\partial S^2} (dS_t)^2$$

Let's calculate the partial derivatives for our specific function f :

$$\frac{\partial f}{\partial t} = 0$$

$$\frac{\partial f}{\partial S} = \frac{1}{S_t}$$

$$\frac{\partial^2 f}{\partial S^2} = -\frac{1}{S_t^2}$$

Substituting these derivatives back into Itô's formula yields:

$$d(\ln(S_t)) = \frac{1}{S_t} dS_t + \frac{1}{2} \left(-\frac{1}{S_t^2} (dS_t)^2 \right)$$

To evaluate the squared differential term $(dS_t)^2$, let's square our original SDE expression:

$$(dS_t)^2 = (\mu S_t dt + \sigma S_t dW_t)^2$$

$$(dS_t)^2 = \mu^2 S_t^2 (dt)^2 + 2\mu\sigma S_t^2 dt dW_t + \sigma^2 S_t^2 (dW_t)^2$$

Knowing the following properties:

- $dt \cdot dt \rightarrow 0$
- $dt \cdot dW_t \rightarrow 0$
- $dW_t \cdot dW_t \rightarrow dt$

We finally get:

$$(dS_t)^2 = \sigma^2 S_t^2 dt$$

By substituting our original expression for dS_t and our newly calculated $(dS_t)^2$ into the equation, we have:

$$d(\ln(S_t)) = \frac{1}{S_t} (\mu S_t dt + \sigma S_t dW_t) - \frac{1}{2 S_t^2} (\sigma^2 S_t^2 dt)$$

We distribute the terms across the fractions:

$$d(\ln(S_t)) = \mu dt + \sigma dW_t - \frac{1}{2} \sigma^2 dt$$

$$\Leftrightarrow d(\ln(S_t)) = \left(\mu - \frac{\sigma^2}{2} \right) dt + \sigma dW_t$$

We can directly integrate both sides from the initial time ($t = 0$) to our final maturity horizon ($t = T$):

$$\begin{aligned} \int_0^T d(\ln S_t) &= \int_0^T \left(\mu - \frac{\sigma^2}{2} \right) dt + \int_0^T \sigma dW_t \\ \Leftrightarrow \ln(S_T) - \ln(S_0) &= \left(\mu - \frac{\sigma^2}{2} \right) T + \sigma(W_T - W_0) \\ \Leftrightarrow \ln\left(\frac{S_T}{S_0}\right) &= \left(\mu - \frac{\sigma^2}{2} \right) T + \sigma W_T \\ \Leftrightarrow S_T = S_0 \exp\left(\left(\mu - \frac{\sigma^2}{2}\right) T + \sigma W_T\right) \end{aligned}$$

Appendix 3. — Student's t-distribution

We want to show that:

$$\lim_{\nu \rightarrow \infty} f(x) = \frac{1}{\sqrt{2\pi}} e^{-\frac{x^2}{2}}$$

. The PDF is given by:

$$f(x) = \frac{\Gamma(\frac{\nu+1}{2})}{\sqrt{\nu\pi}\Gamma(\frac{\nu}{2})} \left(1 + \frac{x^2}{\nu}\right)^{-\frac{\nu+1}{2}}$$

If we analyze this into two independent parts, and using the fundamental definition of the exponential constant $\lim_{n \rightarrow +\infty} (1 + \frac{a}{n})^n = e^a$, we evaluate the limit of the right-hand term:

$$\lim_{\nu \rightarrow \infty} \left(1 + \frac{x^2}{\nu}\right)^{-\frac{\nu+1}{2}} = \lim_{\nu \rightarrow \infty} \left[\left(1 + \frac{x^2}{\nu}\right)^\nu\right]^{-\frac{1}{2}} \cdot \left(1 + \frac{x^2}{\nu}\right)^{-\frac{1}{2}}$$

Thus, the variable part yields exactly the Gaussian kernel: $e^{-\frac{x^2}{2}}$

Using the Strling's approximation for the Gamma function, which states that for large z :

$$\Gamma(z) \sim \sqrt{\frac{2\pi}{z}} \left(\frac{z}{e}\right)^z$$

Let's apply this to the ratio $\frac{\Gamma(\frac{\nu+1}{2})}{\Gamma(\frac{\nu}{2})}$ by setting $z = \frac{\nu}{2}$:

$$\begin{aligned} \frac{\Gamma(z + \frac{1}{2})}{\Gamma(z)} &\sim \frac{\sqrt{\frac{2\pi}{z+1/2}} \left(\frac{z+1/2}{e}\right)^{z+1/2}}{\sqrt{\frac{2\pi}{z}} \left(\frac{z}{e}\right)^z} \\ &= \sqrt{\frac{z}{z+1/2}} \cdot e^{-1/2} \cdot \frac{(z+1/2)^{z+1/2}}{z^z} \end{aligned}$$

Rearranging the last fraction:

$$\frac{(z+1/2)^z \cdot (z+1/2)^{1/2}}{z^z} = \left(1 + \frac{1}{2z}\right)^z \cdot \sqrt{z+1/2}$$

Taking the limit as $z \rightarrow \infty$:

- $\sqrt{\frac{z}{z+1/2}} \rightarrow 1$
- $\left(1 + \frac{1}{2z}\right)^z \rightarrow e^{1/2}$

Hence:

$$\lim_{z \rightarrow \infty} \frac{\Gamma(z + \frac{1}{2})}{\Gamma(z)} = \sqrt{z + \frac{1}{2}} \sim \sqrt{z} = \sqrt{\frac{\nu}{2}}$$

If we substitute this back into the normalization constant multiplier:

$$\lim_{\nu \rightarrow \infty} \frac{1}{\sqrt{\nu\pi}} \frac{\Gamma(\frac{\nu+1}{2})}{\Gamma(\frac{\nu}{2})} = \frac{1}{\sqrt{\nu\pi}} \cdot \sqrt{\frac{\nu}{2}} = \frac{1}{\sqrt{2\pi}}$$

Combining both parts gives the standard normal density function:

$$f(x) \rightarrow \frac{1}{\sqrt{2\pi}} e^{-\frac{x^2}{2}}$$

Let T_ν be a standard Student's t-distributed random variable with ν degrees of freedom. By definition: $\mathbb{E}[T_\nu] = 0$ and $\mathbb{V}(T_\nu) = \frac{\nu}{\nu-2}$ for $\nu > 2$.

To scale a random variable so that its variance equals 1, we must divide by its standard deviation:

$$Z = \frac{T_\nu}{\sqrt{\mathbb{V}(T_\nu)}} = \frac{T_\nu}{\sqrt{\frac{\nu}{\nu-2}}} = T_\nu \sqrt{\frac{\nu-2}{\nu}}$$

Now Z represents a zero-mean, unit-variance process containing the heavy tails dictated by ν .

Under continuous geometric scaling over a time horizon T , with an expected annualized return μ and annualized volatility σ , the total return volatility scales by $\sqrt{T}$. We inject Z into the return dynamics:

$$\begin{aligned} \ln\left(\frac{S_T}{S_0}\right) &= \mu T + \sigma \sqrt{T} Z \\ \Leftrightarrow \ln\left(\frac{S_T}{S_0}\right) &= \mu T + \sigma \sqrt{T} \left(T_\nu \sqrt{\frac{\nu-2}{\nu}}\right) \\ \Leftrightarrow \ln\left(\frac{S_T}{S_0}\right) &= \mu T + \sigma \sqrt{\frac{\nu-2}{\nu}} T_\nu \sqrt{T} \\ \Leftrightarrow S_T = S_0 \exp\left(\mu T + \sigma \sqrt{\frac{\nu-2}{\nu}} T_\nu \sqrt{T}\right) \end{aligned}$$

Appendix 4. — Python code

```
1#!/usr/bin/env python3
2# -*- coding: utf-8 -*-
3"""
4Created on Sun June 7 15:03:00 2026
5
6@author: Quantitative Risk Management Framework
7"""
8
9import numpy as np
10import scipy.stats as stats
11import matplotlib.pyplot as plt
12import yfinance as yf
13
14# Set global seed for exact reproducibility across all
15# random generations
16np.random.seed(42)
17
18# =====
19# 1. MARKET DATA ACQUISITION & PORTFOLIO CONSTRUCTION
20# =====
21print("FETCHING REAL-WORLD MARKET DATA VIA YFINANCE...")
```



```

21 # Define tickers for the requested global indices
22 # ^GSPC: S&P 500, ^IXIC: NASDAQ, ^FCHI: CAC 40, ^FTSE:
23   ↳ FTSE 100, ^GDAXI: DAX, ^HSI: Hang Seng
24 tickers = {
25     'S&P 500': '^GSPC',
26     'NASDAQ': '^IXIC',
27     'CAC 40': '^FCHI',
28     'FTSE 100': '^FTSE',
29     'DAX': '^GDAXI',
30     'Hang Seng': '^HSI'
31 }
32
33 # Fetch 5 years of historical data to capture full market
34   ↳ cycles/regimes
35 start_date = "2021-06-01"
36 end_date = "2026-06-01"
37
38 data = yf.download(list(tickers.values()),
39   ↳ start=start_date, end=end_date)['Close']
40 data = data.dropna() # Drop non-overlapping
41   ↳ holidays/weekends across global exchanges
42
43 # Compute daily log returns for statistical symmetry
44 asset_returns = np.log(data / data.shift(1)).dropna()
45
46 # Construct an equally-weighted portfolio
47 num_assets = len(tickers)
48 weights = np.array([1.0 / num_assets] * num_assets)
49
50 # Generate the historical portfolio returns time series
51 historical_returns = asset_returns.dot(weights).to_numpy()
52 n_days = len(historical_returns)
53
54 # Extract empirical parameters for log printout
55 emp_drift = np.mean(historical_returns)
56 emp_volatility = np.std(historical_returns)
57
58 # =====
59 # 2. USER-DEFINED SIMULATION INPUT PARAMETERS
60 # =====
61 M = 50000 # Number of Monte Carlo
62   ↳ scenarios
63 alpha = 0.975 # Confidence level for baseline
64   ↳ CDF plotting (Plot 2)
65
66 initial_price = 100 # Starting asset price for
67   ↳ trajectory paths simulation
68 n_path_steps = 100 # Horizon length (in days) to
69   ↳ project the sample paths
70 n_paths_to_plot = 15 # Number of random paths to
71   ↳ visualize per method
72
73 print("\nINITIALIZING PORTFOLIO STRESS TESTING ENGINE...")
74 print(f"Historical Sample Size: {n_days} trading days")
75 print(f"Empirical Daily Drift (Mean): {emp_drift:.4%}")
76 print(f"Empirical Daily Volatility (Std Dev):
77   ↳ {emp_volatility:.4%}\n" + "-"*60)
78
79 # =====
80 # APPROACH 1: GEOMETRIC BROWNIAN MOTION (NORMAL)
81 # =====
82 # GBM extracts parameters via Maximum Likelihood
83   ↳ Estimation, assuming perfect normality
84 mu_norm, sigma_norm = stats.norm.fit(historical_returns)
85
86 # 1A. Returns for VaR/CVaR static horizon
87 sim_returns_gbm = np.random.normal(loc=mu_norm,
88   ↳ scale=sigma_norm, size=M)
89
90 # 1B. Dynamic path generation matrix (steps x paths)
91 paths_returns_gbm = np.random.normal(loc=mu_norm,
92   ↳ scale=sigma_norm, size=(n_path_steps,
93   ↳ n_paths_to_plot))
94 price_paths_gbm = initial_price *
95   ↳ np.exp(np.vstack([np.zeros(n_paths_to_plot),
96   ↳ np.cumsum(paths_returns_gbm, axis=0)]))
97
98 # =====
99 # APPROACH 2: STUDENT'S T-DISTRIBUTION
100 # =====
101 # Fits parameters directly to map history onto a
102   ↳ theoretical t-curve (captures fat tails)
103
104 df_est, loc_est, scale_est =
105   ↳ stats.t.fit(historical_returns)
106
107 # 2A. Returns for VaR/CVaR static horizon
108 sim_returns_t = stats.t.rvs(df=df_est, loc=loc_est,
109   ↳ scale=scale_est, size=M)
110
111 # 2B. Dynamic path generation matrix
112 paths_returns_t = stats.t.rvs(df=df_est, loc=loc_est,
113   ↳ scale=scale_est, size=(n_path_steps, n_paths_to_plot))
114 price_paths_t = initial_price *
115   ↳ np.exp(np.vstack([np.zeros(n_paths_to_plot),
116   ↳ np.cumsum(paths_returns_t, axis=0)]))
117
118 # =====
119 # APPROACH 3: KERNEL DENSITY ESTIMATION (KDE)
120 # =====
121 # Fits a non-parametric Gaussian KDE over the historical
122   ↳ sample returns
123 kde = stats.gaussian_kde(historical_returns,
124   ↳ bw_method='silverman')
125 h = kde.factor * np.std(historical_returns)
126
127 # 3A. Returns for VaR/CVaR static horizon (Hierarchical
128   ↳ sampling)
129 random_indices = np.random.choice(n_days, size=M,
130   ↳ replace=True)
131 x_I = historical_returns[random_indices]
132 Z = np.random.normal(0, 1, M)
133 sim_returns_kde = x_I + h * Z
134
135 # 3B. Dynamic path generation matrix via structured KDE
136   ↳ drawing loops
137 paths_returns_kde = np.zeros((n_path_steps,
138   ↳ n_paths_to_plot))
139 for step in range(n_path_steps):
140   idx = np.random.choice(n_days, size=n_paths_to_plot,
141     ↳ replace=True)
142   paths_returns_kde[step, :] = historical_returns[idx] +
143     ↳ h * np.random.normal(0, 1, n_paths_to_plot)
144 price_paths_kde = initial_price *
145   ↳ np.exp(np.vstack([np.zeros(n_paths_to_plot),
146   ↳ np.cumsum(paths_returns_kde, axis=0)]))
147
148 # =====
149 # RISK METRICS EVALUATION FUNCTION
150 # =====
151 def calculate_var_cvar(sim_returns, alpha=0.975):
152     """
153     Sorts returns and extracts Value at Risk and
154     ↳ Conditional Value at Risk.
155     Returns are transformed to positive numbers
156     ↳ representing losses.
157     """
158     sorted_returns = np.sort(sim_returns)
159     k = int(np.floor((1 - alpha) * len(sorted_returns)))
160
161     var = -sorted_returns[k]
162     cvar = -np.mean(sorted_returns[:k])
163     return var, cvar
164
165 # =====
166 # COMPUTE SYNCHRONIZED TARGET METRICS
167 # =====
168 # Calculate specific cross-quantile targets for Plot 1 and
169   ↳ Plot 3
170 var_gbm_99, _ = calculate_var_cvar(sim_returns_gbm,
171   ↳ alpha=0.99)
172 var_t_99, _ = calculate_var_cvar(sim_returns_t,
173   ↳ alpha=0.99)
174 var_kde_99, _ = calculate_var_cvar(sim_returns_kde,
175   ↳ alpha=0.99)
176
177 _, cvar_gbm_975 = calculate_var_cvar(sim_returns_gbm,
178   ↳ alpha=0.975)
179 _, cvar_t_975 = calculate_var_cvar(sim_returns_t,
180   ↳ alpha=0.975)
181 _, cvar_kde_975 = calculate_var_cvar(sim_returns_kde,
182   ↳ alpha=0.975)
183
184 # Calculate Skewness and Excess Kurtosis across frameworks

```

```

149 skew_gbm, kurt_gbm = stats.skew(sim_returns_gbm),
    ↳ stats.kurtosis(sim_returns_gbm)
150 skew_t, kurt_t = stats.skew(sim_returns_t),
    ↳ stats.kurtosis(sim_returns_t)
151 skew_kde, kurt_kde = stats.skew(sim_returns_kde),
    ↳ stats.kurtosis(sim_returns_kde)
152
153 # =====
154 # STATISTICAL TESTS FOR SKEWNESS AND KURTOSIS
155 # =====
156 def test_skewness_kurtosis(data, name):
157     """
158     Perform statistical tests for significance of skewness
159     ↳ and excess kurtosis.
160     """
161     n = len(data)
162     sample_skew = stats.skew(data)
163     sample_kurt = stats.kurtosis(data) # Excess kurtosis
164
165     if n < 4:
166         return {
167             'name': name, 'skew': sample_skew, 'kurt':
168             ↳ sample_kurt,
169             'skew_pvalue': np.nan, 'kurt_pvalue': np.nan,
170             ↳ 'dagostino_pvalue': np.nan
171         }
172
173     se_skew = np.sqrt(6 * n * (n - 1) / ((n - 2) * (n + 1)
174     ↳ * (n + 3)))
175     se_kurt = np.sqrt(24 * n * (n - 1)**2 / ((n - 3) * (n
176     ↳ - 2) * (n + 1) * (n + 3)))
177
178     z_skew = sample_skew / se_skew if not
179     ↳ np.isnan(se_skew) else np.nan
180     z_kurt = sample_kurt / se_kurt if not
181     ↳ np.isnan(se_kurt) else np.nan
182
183     p_skew = 2 * (1 - stats.norm.cdf(abs(z_skew))) if not
184     ↳ np.isnan(z_skew) else np.nan
185     p_kurt = 2 * (1 - stats.norm.cdf(abs(z_kurt))) if not
186     ↳ np.isnan(z_kurt) else np.nan
187
188     dagostino_stat, dagostino_p = stats.normaltest(data)
189
190     return {
191         'name': name, 'skew': sample_skew, 'kurt':
192         ↳ sample_kurt,
193         'z_skew': z_skew, 'z_kurt': z_kurt,
194         'skew_pvalue': p_skew, 'kurt_pvalue': p_kurt,
195         'dagostino_stat': dagostino_stat,
196         ↳ 'dagostino_pvalue': dagostino_p
197     }
198
199 # Perform tests for each distribution
200 results_gbm = test_skewness_kurtosis(sim_returns_gbm, 'GBM
201 ↳ (Normal)')
202 results_t = test_skewness_kurtosis(sim_returns_t,
203 ↳ "Student's t")
204 results_kde = test_skewness_kurtosis(sim_returns_kde,
205 ↳ 'KDE')
206
207 # =====
208 # RESULTS PRINT OUT
209 # =====
210 print(f"{'Approach':<30} | {'99.0% VaR':<10} | {'97.5%
211 ↳ CVaR':<10} | {'Skewness':<9} | {'Ex. Kurt':<9}")
212 print("-" * 78)
213 print(f"{'1. Geometric Brownian (Normal)':<30} |
214 ↳ {var_gbm_99:.4%} | {cvar_gbm_975:.4%} |
215 ↳ {skew_gbm:+0.4f} | {kurt_gbm:+0.4f}")
216 print(f"{'2. Student's t-Distribution':<30} |
217 ↳ {var_t_99:.4%} | {cvar_t_975:.4%} | {skew_t:+0.4f}
218 ↳ | {kurt_t:+0.4f}")
219 print(f"{'3. Kernel Density Estimator':<30} |
220 ↳ {var_kde_99:.4%} | {cvar_kde_975:.4%} |
221 ↳ {skew_kde:+0.4f} | {kurt_kde:+0.4f}")
222 print("-" * 78 + "\n")
223
224 # =====
225 # PRINT STATISTICAL TEST RESULTS
226 # =====
227
228 print("\n" + "=" * 80)
229 print("STATISTICAL SIGNIFICANCE TESTS FOR SKEWNESS AND
230 ↳ KURTOSIS")
231 print("=" * 80)
232
233 for results in [results_gbm, results_t, results_kde]:
234     skew_sig = '***' if results['skew_pvalue'] < 0.001
235     ↳ else '**' if results['skew_pvalue'] < 0.01 else ''
236     kurt_sig = '***' if results['kurt_pvalue'] < 0.001
237     ↳ else '**' if results['kurt_pvalue'] < 0.01 else ''
238     dag_sig = '***' if results['dagostino_pvalue'] < 0.001
239     ↳ else '**' if results['dagostino_pvalue'] < 0.01
240     ↳ else '' if results['dagostino_pvalue'] < 0.05
241     ↳ else ''
242
243     print(f"\n{results['name']}:")
244     print(f" Skewness: {results['skew']:+0.4f} (z =
245     ↳ {results['z_skew']:+0.4f}, p-value =
246     ↳ {results['skew_pvalue']:.6f} {skew_sig})")
247     print(f" Ex. Kurtosis: {results['kurt']:+0.4f} (z =
248     ↳ {results['z_kurt']:+0.4f}, p-value =
249     ↳ {results['kurt_pvalue']:.6f} {kurt_sig})")
250     print(f" D'Agostino K^2: stat =
251     ↳ {results['dagostino_stat']:+0.4f}, p-value =
252     ↳ {results['dagostino_pvalue']:.6f} {dag_sig}")
253
254 print("\n" + "-" * 80)
255 print("Significance codes: '***' p < 0.001, '**' p < 0.01,
256 ↳ '*' p < 0.05")
257 print("=" * 80 + "\n")
258
259 global_min_tail = np.percentile(historical_returns, 0.1)
260 global_max_tail = np.percentile(historical_returns, 15)
261
262 # Global styling mapping for the comparative charts
263 colors = {'gbm': '#1f77b4', 't': '#ff7f0e', 'kde':
264 ↳ '#2ca02c'}
265
266 # =====
267 # PLOT 1: PROBABILITY DENSITY TAIL FOCUS
268 # =====
269 plt.figure(figsize=(10, 6))
270 plt.hist(sim_returns_gbm, bins=250, density=True,
271 ↳ alpha=0.3, color=colors['gbm'], label='GBM (Normal)')
272 plt.hist(sim_returns_t, bins=250, density=True, alpha=0.3,
273 ↳ color=colors['t'], label="Student's t")
274 plt.hist(sim_returns_kde, bins=250, density=True,
275 ↳ alpha=0.3, color=colors['kde'], label='KDE')
276
277 plt.axvline(-var_gbm_99, color=colors['gbm'],
278 ↳ linestyle=':', linewidth=2, label=f'GBM 99% VaR
279 ↳ ({var_gbm_99:.2%})')
280 plt.axvline(-var_t_99, color=colors['t'], linestyle=':',
281 ↳ linewidth=2, label=f'Student t 99% VaR
282 ↳ ({var_t_99:.2%})')
283 plt.axvline(-var_kde_99, color=colors['kde'],
284 ↳ linestyle=':', linewidth=2, label=f'KDE 99% VaR
285 ↳ ({var_kde_99:.2%})')
286
287 plt.axvline(-cvar_gbm_975, color=colors['gbm'],
288 ↳ linestyle='-', linewidth=2, label=f'GBM 97.5% CVaR
289 ↳ ({cvar_gbm_975:.2%})')
290 plt.axvline(-cvar_t_975, color=colors['t'], linestyle='-',
291 ↳ linewidth=2, label=f'Student t 97.5% CVaR
292 ↳ ({cvar_t_975:.2%})')
293 plt.axvline(-cvar_kde_975, color=colors['kde'],
294 ↳ linestyle='-', linewidth=2, label=f'KDE 97.5% CVaR
295 ↳ ({cvar_kde_975:.2%})')
296
297 plt.title("Probability Density Focus: 99% VaR vs 97.5%
298 ↳ CVaR Loss Tail (Real Data)", fontsize=13,
299 ↳ fontweight='bold')
300 plt.xlabel("Daily Portfolio Returns")
301 plt.ylabel("Density")
302 plt.xlim(global_min_tail, global_max_tail)
303 plt.legend(fontsize=9, loc='upper left')
304 plt.grid(True, alpha=0.2)
305 plt.show()
306
307 # =====
308 # PLOT 2: CUMULATIVE DISTRIBUTION FUNCTION (CDF)
309 # =====
310 plt.figure(figsize=(10, 6))

```

```

262 for data, name, col in zip([sim_returns_gbm,
    ↪ sim_returns_t, sim_returns_kde],
263                             ['GBM (Normal)', "Student's t",
    ↪ 'KDE'],
264                             [colors['gbm'], colors['t'],
    ↪ colors['kde']]):
265     sorted_data = np.sort(data)
266     cdf = np.arange(1, len(sorted_data) + 1) /
    ↪ len(sorted_data)
267     plt.plot(sorted_data, cdf, label=name, color=col,
    ↪ linewidth=2)
268
269 plt.axhline(1-0.975, color='red', linestyle='--',
    ↪ linewidth=1.2, label='Significance Level (0.025)')
270 plt.axhline(1-0.99, color='blue', linestyle='--',
    ↪ linewidth=1.2, label='Significance Level (0.01)')
271 plt.title("Empirical CDF Quantile Trajectory (Real Data)",
    ↪ fontsize=13, fontweight='bold')
272 plt.xlabel("Daily Portfolio Returns")
273 plt.ylabel("Cumulative Probability")
274 plt.xlim(np.percentile(historical_returns, 0.1), 0.01)
275 plt.ylim(0, 0.10)
276 plt.legend(fontsize=10, loc='lower right')
277 plt.grid(True, alpha=0.2)
278 plt.show()
279
280 # =====
281 # PLOT 3: DIRECT METRIC COMPARISON (BAR CHART)
282 # =====
283 fig, ax = plt.subplots(figsize=(10, 6))
284 categories = ['GBM (Normal)', "Student's t", 'KDE']
285 var_values = [var_gbm_99 * 100, var_t_99 * 100, var_kde_99
    ↪ * 100]
286 cvar_values = [cvar_gbm_975 * 100, cvar_t_975 * 100,
    ↪ cvar_kde_975 * 100]
287
288 x = np.arange(len(categories))
289 width = 0.35
290 rects1 = ax.bar(x - width/2, var_values, width, label='99%
    ↪ VaR', color='#bcbd22', alpha=0.85)
291 rects2 = ax.bar(x + width/2, cvar_values, width,
    ↪ label='97.5% CVaR', color='#d62728', alpha=0.85)
292
293 ax.set_title("Direct Value Comparison: 99% VaR vs 97.5%
    ↪ CVaR (%)", fontsize=13, fontweight='bold')
294 ax.set_xticks(x)
295 ax.set_xticklabels(categories)
296 ax.set_ylabel("Loss Scale (%)")
297 ax.legend(fontsize=10, loc='upper left')
298 ax.grid(True, axis='y', alpha=0.3)
299
300 def autolabel(rects):
301     for rect in rects:
302         height = rect.get_height()
303         ax.annotate(f'{height:.2f}%',
304                     xy=(rect.get_x() + rect.get_width() /
    ↪ 2, height),
305                     xytext=(0, 3), textcoords="offset
    ↪ points",
306                     ha='center', va='bottom', fontsize=9,
    ↪ fontweight='bold')
307
308 autolabel(rects1)
309 autolabel(rects2)
310 plt.tight_layout()
311 plt.show()
312
313 # =====
314 # PLOTS 4, 5, 6: SIMULATED PRICE TRAJECTORIES
315 # =====
316 for paths, name, col in zip([price_paths_gbm,
    ↪ price_paths_t, price_paths_kde],
317                             ['Geometric Brownian Motion
    ↪ (Normal)', "Student's
    ↪ t-Distribution", 'Kernel
    ↪ Density Estimator (KDE)'],
318                             [colors['gbm'], colors['t'],
    ↪ colors['kde']]):
319     plt.figure(figsize=(10, 4))
320     plt.plot(paths, linewidth=1.5, color=col, alpha=0.6)
321     plt.title(f"{name} Simulated Price Paths (Real Data)",
    ↪ fontsize=12, fontweight='bold')
322     plt.xlabel("Time Horizon (Days)")
323     plt.ylabel("Asset Price")
324     plt.grid(True, alpha=0.3)
325
326     plt.axhline(initial_price, color='black',
    ↪ linestyle='--', alpha=0.5)
327     plt.show()
328
329 # =====
330 # PREPARATION FOR DISTRIBUTION OVERLAYS (PLOTS 7-9)
331 # =====
332 x_axis = np.linspace(np.min(historical_returns),
    ↪ np.max(historical_returns), 1000)
333 normal_curve = stats.norm.pdf(x_axis, loc=mu_norm,
    ↪ scale=sigma_norm)
334
335 # PLOT 7: HISTOGRAM - GBM
336 plt.figure(figsize=(10, 4))
337 plt.hist(sim_returns_gbm, bins=150, density=True,
    ↪ alpha=0.6, color=colors['gbm'], edgecolor='black',
    ↪ linewidth=0.5)
338 plt.plot(x_axis, normal_curve, color='red', linestyle='-',
    ↪ linewidth=2, label='Theoretical Normal Curve')
339 plt.title("Distribution of Returns - Geometric Brownian
    ↪ Motion (Normal)", fontsize=12, fontweight='bold')
340 plt.xlabel("Simulated Daily Returns")
341 plt.ylabel("Frequency Density")
342 plt.xlim(np.percentile(historical_returns, 0.1),
    ↪ np.percentile(historical_returns, 99.9))
343 plt.legend()
344 plt.grid(True, alpha=0.2)
345 plt.show()
346
347 # PLOT 8: HISTOGRAM - STUDENT'S T
348 plt.figure(figsize=(10, 4))
349 plt.hist(sim_returns_t, bins=150, density=True, alpha=0.6,
    ↪ color=colors['t'], edgecolor='black', linewidth=0.5)
350 plt.plot(x_axis, normal_curve, color='red', linestyle='-',
    ↪ linewidth=2, label='Theoretical Normal Curve')
351 t_curve = stats.t.pdf(x_axis, df=df_est, loc=loc_est,
    ↪ scale=scale_est)
352 plt.plot(x_axis, t_curve, color='black', linestyle='--',
    ↪ linewidth=2, label='Fitted Student's t Curve')
353 plt.title("Distribution of Returns - Student's
    ↪ t-Distribution", fontsize=12, fontweight='bold')
354 plt.xlabel("Simulated Daily Returns")
355 plt.ylabel("Frequency Density")
356 plt.xlim(np.percentile(historical_returns, 0.1),
    ↪ np.percentile(historical_returns, 99.9))
357 plt.legend()
358 plt.grid(True, alpha=0.2)
359 plt.show()
360
361 # PLOT 9: HISTOGRAM - KDE
362 plt.figure(figsize=(10, 4))
363 plt.hist(sim_returns_kde, bins=150, density=True,
    ↪ alpha=0.6, color=colors['kde'], edgecolor='black',
    ↪ linewidth=0.5)
364 plt.plot(x_axis, normal_curve, color='red', linestyle='-',
    ↪ linewidth=2, label='Theoretical Normal Curve')
365 plt.plot(x_axis, kde.pdf(x_axis), color='darkgreen',
    ↪ linestyle='-', linewidth=2, label='KDE Continuous Fit
    ↪ Line')
366 plt.title("Distribution of Returns - Kernel Density
    ↪ Estimator (KDE)", fontsize=12, fontweight='bold')
367 plt.xlabel("Simulated Daily Returns")
368 plt.ylabel("Frequency Density")
369 plt.xlim(np.percentile(historical_returns, 0.1),
    ↪ np.percentile(historical_returns, 99.9))
370 plt.legend()
371 plt.grid(True, alpha=0.2)
372 plt.show()

```

```

1 #!/usr/bin/env python3
2
3 import numpy as np
4 import scipy.stats as stats
5 import matplotlib.pyplot as plt
6 import yfinance as yf
7 import warnings
8 warnings.filterwarnings('ignore')
9
10 np.random.seed(42)
11
12 # =====
13 # 1. Data Retrieval (Same portfolio as original paper)
14 # =====
15 print("Fetching data...")
16

```

```

17 tickers = ['^GSPC', '^IXIC', '^FCHI', '^FTSE', '^GDAXI',
18           ↪ '^HSI']
19 names = ['S&P500', 'NASDAQ', 'CAC40', 'FTSE100', 'DAX',
20           ↪ 'HangSeng']
21
22 # Original paper: 2021-2026
23 # Split: 2015-2020 for training, 2021-2024 for validation
24 data_all = yf.download(tickers, start="2015-01-01",
25                       ↪ end="2024-12-31", auto_adjust=True)['Close']
26 data_all = data_all.dropna()
27
28 log_returns_all = np.log(data_all /
29                       ↪ data_all.shift(1)).dropna()
30 weights = np.array([1/6] * 6)
31 portfolio_returns_all =
32 ↪ log_returns_all.dot(weights).to_numpy()
33 dates_all = log_returns_all.index
34
35 # Training period (for model fitting)
36 train_mask = log_returns_all.index < '2021-01-01'
37 train_returns = portfolio_returns_all[train_mask]
38
39 # Validation period (for backtesting)
40 test_mask = log_returns_all.index >= '2021-01-01'
41 test_returns = portfolio_returns_all[test_mask]
42 test_dates = dates_all[test_mask]
43
44 print(f"Training period: 2015-2020 ({len(train_returns)}
45 ↪ days)")
46 print(f"Validation period: 2021-2024 ({len(test_returns)}
47 ↪ days)")
48
49 # =====
50 # 2. Model Fitting on Training Data
51 # =====
52 M = 50000
53
54 # GBM
55 mu_norm, sigma_norm = stats.norm.fit(train_returns)
56 sim_gbm = np.random.normal(mu_norm, sigma_norm, M)
57
58 # Student's t
59 df_t, loc_t, scale_t = stats.t.fit(train_returns)
60 sim_t = stats.t.rvs(df=df_t, loc=loc_t, scale=scale_t,
61 ↪ size=M)
62
63 # KDE
64 kde = stats.gaussian_kde(train_returns,
65 ↪ bw_method='silverman')
66 h = kde.factor * np.std(train_returns)
67 idx = np.random.choice(len(train_returns), size=M,
68 ↪ replace=True)
69 sim_kde = train_returns[idx] + h * np.random.normal(0, 1,
70 ↪ M)
71
72 print(f"Student's t degrees of freedom: {df_t:.2f}")
73
74 # =====
75 # 3. VaR/CVaR Calculation
76 # =====
77 def calc_var_cvar(sim, alpha):
78     sorted_r = np.sort(sim)
79     k = int(np.floor((1 - alpha) * len(sorted_r)))
80     var = -sorted_r[k]
81     cvar = -np.mean(sorted_r[k:])
82     return var, cvar
83
84 var_gbm, cvar_gbm = calc_var_cvar(sim_gbm, 0.99),
85 ↪ calc_var_cvar(sim_gbm, 0.975)
86 var_t, cvar_t = calc_var_cvar(sim_t, 0.99),
87 ↪ calc_var_cvar(sim_t, 0.975)
88 var_kde, cvar_kde = calc_var_cvar(sim_kde, 0.99),
89 ↪ calc_var_cvar(sim_kde, 0.975)
90
91 var_gbm_99, cvar_gbm_975 = var_gbm[0], cvar_gbm[1]
92 var_t_99, cvar_t_975 = var_t[0], cvar_t[1]
93 var_kde_99, cvar_kde_975 = var_kde[0], cvar_kde[1]
94
95 print(f"\n{'Model':<15} {'VaR(99%)':<12} {'CVaR(97.5%)':<12}")
96 print(f"{'-'*40}")
97 print(f"{'GBM':<15} {var_gbm_99:.4%}
98 ↪ {cvar_gbm_975:.4%}")
99 print(f"{'Student t':<15} {var_t_99:.4%}
100 ↪ {cvar_t_975:.4%}")
101 print(f"{'KDE':<15} {var_kde_99:.4%}
102 ↪ {cvar_kde_975:.4%}")
103
104 # =====
105 # 4. Backtest (Validation period 2021-2024)
106 # =====
107 print("\n=== Backtest Results (2021-2024) ===")
108 print(f"Theoretical: VaR(99%) violations should occur at 1%
109 ↪ frequency\n")
110
111 viol_gbm = test_returns < -var_gbm_99
112 viol_t = test_returns < -var_t_99
113 viol_kde = test_returns < -var_kde_99
114
115 rate_gbm = np.mean(viol_gbm)
116 rate_t = np.mean(viol_t)
117 rate_kde = np.mean(viol_kde)
118
119 def judge(rate):
120     if rate > 0.015:
121         return "Underestimating risk (Dangerous)"
122     elif rate < 0.005:
123         return "Too conservative"
124     else:
125         return "Appropriate"
126
127 print(f"GBM violation rate: {rate_gbm:.2%}
128 ↪ {judge(rate_gbm)}")
129 print(f"Student t violation rate: {rate_t:.2%}
130 ↪ {judge(rate_t)}")
131 print(f"KDE violation rate: {rate_kde:.2%}
132 ↪ {judge(rate_kde)}")
133
134 # =====
135 # 5. Major Event Validation
136 # =====
137 print("\n=== Validation on Major Market Events ===")
138
139 events = {
140     'COVID Crash (2020/3)': '2020-03-16',
141     'Russia Invasion (2022/2)': '2022-02-24',
142     'Fed Rate Shock (2022/6)': '2022-06-13',
143     'SVB Collapse (2023/3)': '2023-03-13',
144 }
145
146 port_series = log_returns_all.dot(weights)
147
148 for event_name, date in events.items():
149     try:
150         nearby = port_series[date:date]
151         if len(nearby) == 0:
152             nearby =
153             ↪ port_series.iloc[port_series.index.get_indexer([date],
154             ↪ method='nearest')]
155         val = nearby.iloc[0]
156         gbm_breach = "Exceeded" if val < -var_gbm_99 else
157         ↪ "OK"
158         t_breach = "Exceeded" if val < -var_t_99 else
159         ↪ "OK"
160         kde_breach = "Exceeded" if val < -var_kde_99 else
161         ↪ "OK"
162         print(f"{event_name}: {val:.2%} | GBM:{gbm_breach}
163 ↪ | t:{t_breach} | KDE:{kde_breach}")
164     except Exception as e:
165         print(f"{event_name}: No data available")
166
167 # =====
168 # 6. Visualization
169 # =====
170 fig, axes = plt.subplots(2, 2, figsize=(14, 10))
171 fig.suptitle('VaR/CVaR Backtest (Train:2015-2020,
172 ↪ Test:2021-2024)', fontsize=13, fontweight='bold')
173
174 # Return distribution
175 ax1 = axes[0, 0]
176 ax1.hist(train_returns, bins=100, density=True, alpha=0.5,
177 ↪ color='gray', label='Actual (Train)')
178 x = np.linspace(train_returns.min(), train_returns.max(),
179 ↪ 1000)
180 ax1.plot(x, stats.norm.pdf(x, mu_norm, sigma_norm), 'b-',
181 ↪ lw=2, label='GBM')
182 ax1.plot(x, stats.t.pdf(x, df_t, loc_t, scale_t), 'r-',
183 ↪ lw=2, label='Student t')
184 ax1.plot(x, kde.pdf(x), 'g-', lw=2, label='KDE')
185 ax1.set_title('Return Distribution (Training Period)')
186 ax1.legend(fontsize=8)
187 ax1.set_xlim(-0.08, 0.06)

```

```

156
157 # VaR/CVaR bar chart
158 ax2 = axes[0, 1]
159 models = ['GBM', 'Student t', 'KDE']
160 vars_val = [var_gbm_99*100, var_t_99*100, var_kde_99*100]
161 cvars_val = [cvar_gbm_975*100, cvar_t_975*100,
162             ↪ cvar_kde_975*100]
163 x_pos = np.arange(3)
164 ax2.bar(x_pos - 0.2, vars_val, 0.4, label='VaR(99%)',
165         ↪ color='steelblue', alpha=0.8)
166 ax2.bar(x_pos + 0.2, cvars_val, 0.4, label='CVaR(97.5%)',
167         ↪ color='tomato', alpha=0.8)
168 ax2.set_xticks(x_pos)
169 ax2.set_xticklabels(models)
170 ax2.set_ylabel('Loss Rate (%)')
171 ax2.set_title('VaR/CVaR Comparison')
172 ax2.legend()
173
174 for i, (v, c) in enumerate(zip(vars_val, cvars_val)):
175     ax2.text(i-0.2, v+0.01, f'{v:.2f}%', ha='center',
176             ↪ fontsize=8)
177     ax2.text(i+0.2, c+0.01, f'{c:.2f}%', ha='center',
178             ↪ fontsize=8)
179
180 # Time series with VaR lines
181 ax3 = axes[1, 0]
182 ax3.plot(test_dates, test_returns, color='gray',
183         ↪ alpha=0.5, lw=0.5, label='Returns (Validation)')
184 ax3.axhline(-var_gbm_99, color='blue', lw=1.5,
185             ↪ linestyle='--', label=f'GBM VaR {var_gbm_99:.2%}')
186 ax3.axhline(-var_t_99, color='red', lw=1.5,
187             ↪ linestyle='--', label=f't VaR {var_t_99:.2%}')
188 ax3.axhline(-var_kde_99, color='green', lw=1.5,
189             ↪ linestyle='--', label=f'KDE VaR {var_kde_99:.2%}')
190 breach_dates = test_dates[viol_gbm]
191 ax3.scatter(breach_dates, test_returns[viol_gbm],
192            ↪ color='blue', s=10, alpha=0.7, zorder=5)
193 ax3.set_title('Returns and VaR Violations (Validation
194             ↪ Period)')
195 ax3.legend(fontsize=7)
196
197 # Violation rate bar chart
198 ax4 = axes[1, 1]
199 rates = [rate_gbm*100, rate_t*100, rate_kde*100]
200 colors_b = ['steelblue', 'tomato', 'green']
201 bars = ax4.bar(models, rates, color=colors_b, alpha=0.8)
202 ax4.axhline(1.0, color='black', lw=2, linestyle='-',
203            ↪ label='Theoretical 1.00%')
204 ax4.set_ylabel('Violation Rate (%)')
205 ax4.set_title('VaR(99%) Violation Rate (Validation
206             ↪ Period)')
207 ax4.legend()
208
209 for bar, rate in zip(bars, rates):
210     ax4.text(bar.get_x() + bar.get_width()/2,
211             ↪ bar.get_height() + 0.02,
212             ↪ f'{rate:.2f}%', ha='center', fontsize=10,
213             ↪ fontweight='bold')
214
215 plt.tight_layout()
216 plt.savefig('var_backtest_result.png', dpi=150,
217            ↪ bbox_inches='tight')
218 plt.show()
219 print("\nDone. Saved as var_backtest_result.png")
220
221
222

```